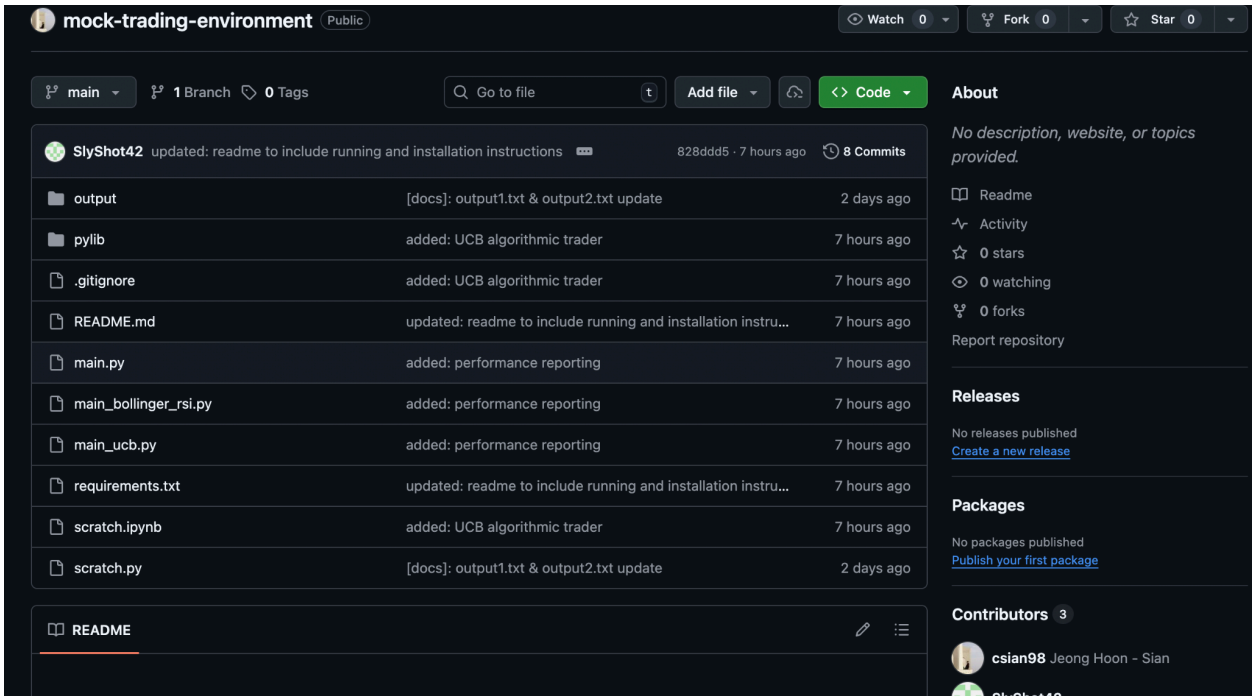
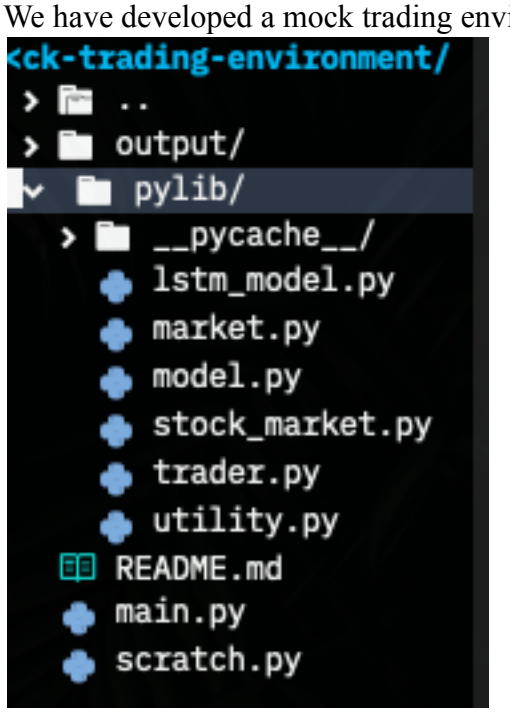
Laboratory Assignment 4

Group 15: Jeong Hoon Choi, Viraj Bansal, Chih-Yun Pai



https://github.com/csian98/mock-trading-environment

We have developed a mock trading environment using the LSTM and <#> methods.



All Python code is stored inside the pylib/ directory, and training and prediction are performed using main.py

```
~/synology-reliquia/academy/USC/MSDS/DSCI-560/laboratory/lab4/mock-trading-environment (main*) » python main.py > output
/output2.txt
```

model.py and market.py each provide implementation declarations.

```python
class StockMarket(Market):
    def __init__(self, symbols: List):
        super().__init__(symbols)

    def history(self, period: str="3y", days: int=30) -> dict:
        output = {}
        for symbol in self._symbols:
            ticker = yf.Ticker(symbol)
            df = ticker.history(period=period)["Close"]
            idx = df.index < (datetime.now() - timedelta(days=days)).strftime("%Y-%m-%d %H:%M:%S%z")
            output[symbol] = df[idx].values
            self._queue[symbol] = deque(df[~idx])

        return output

    def empty(self) -> bool:
        for symbol in self._symbols:
            if len(self._queue[symbol]) == 0:
                return True

        return False

    def next(self) -> dict:
        output = {}
        for symbol in self._symbols:
            output[symbol] = self._queue[symbol].popleft()

        return output
```

The data is provided using a StockMarket class instance. Internally, it uses yfinance to provide data for the specified stock, and provides data for training and daily price predictions through the history and next methods.

```python
class LSTMModel(Model):
    def __init__(self, window_size: int):
        super().__init__("lstm")
        self.window_size = window_size

        self.scaler = MinMaxScaler()

        self.model = Sequential([
            LSTM(units=64, activation="relu", return_sequences=True),
            Dropout(0.3),
            LSTM(units=64, activation="relu", return_sequences=True),
            Dropout(0.3),
            LSTM(units=64, activation="relu", return_sequences=True),
            Dropout(0.3),
            LSTM(units=64, activation="relu", return_sequences=False),
            Dropout(0.3),
            Dense(1)
        ])
        self.model.compile(optimizer=tf.keras.optimizers.Adam(learning_rate=1e-2),
                           loss="mean_squared_error")
```

LSTM used a model with a simple structure consisting of four layers.

```python
class Trader:
    def __init__(self, cash: float):
        self.window_size = 8
        self.symbols = ["NVDA", "AAPL", "GOOG", "MSFT", "AMZN",
                        "META", "TSLA", "AVGO", "ORCL", "NFLX"]
        self.cash = cash
        self.charge = 0.0001

        self.market = StockMarket(self.symbols)
        self.models = {symbol:LSTMModel(self.window_size)\
                       for symbol in self.symbols}

        self.prices = {}
        self.windows = {}
        self.counts = {symbol:0 for symbol in self.symbols}

        history = self.market.history(period="2y", days=45)

        for symbol in self.symbols:
            X, y = window_indexing(history[symbol],
                                   window_size=self.window_size)

            self.prices[symbol] = y[-1]
            self.windows[symbol] = X[-1]
```

All transactions utilize the Trader class.
We used two years of price data for a total of 10 stocks, excluding the last 45 days, for training, and then used the remaining 45 days as simulated trading data.
While no separate fee was charged when buying stocks, a commission of 0.01% of the sale amount was charged when selling stocks.

```
def trade(self):
    scores = {}

    for symbol in self.symbols:
        X = self.windows[symbol]
        pp = self.models[symbol].predict_multiple(
            self.windows[symbol], count=3)
        scores[symbol] = \
            (max(pp) - self.prices[symbol]) / self.prices[symbol]

    for symbol, count in self.counts.items():
        if count != 0 and scores[symbol] < -0.01:
            value = self.prices[symbol] * count
            self.cash += value * (1 - self.charge)
            self.counts[symbol] = 0
        elif count != 0:
            scores[symbol] *= 1.2   # conservation

    candidate = []
    score_sum = 0.0
    for symbol, score in scores.items():
```

We created a strategy based on 3-day price predictions for all symbols every day. If the predicted decline exceeded 1%, sold all holdings. We then weighted the remaining stocks I held and, for stocks with a predicted increase exceeding 3%, diversified our purchases based on the score ratios.

```
def main(*args, **kwargs):
    cash = 100000        # 100 k
    day = 0

    trader = Trader(cash)

    while not trader.empty():
        day += 1
        balance = trader.balance()
        trader.report(day)
        trader.trade()
        trader.next()
```

The scenario starts with $100,000 each time. Then, it calculates and outputs the total assets held each day.

For the Bollinger_RSI model, we use the same data to mock the stock market. The core concept

```python
def get_signal_strength(self, X) -> float:
    '''
    Calculate signal strength based on Bollinger Bands position and RSI.
    Returns a score between -1 and 1:
    - Positive: Oversold condition (buy signal)
    - Negative: Overbought condition (sell signal)
    '''
    prices = X.flatten()
    current_price = prices[-1]

    upper_band, middle_band, lower_band = self._calculate_bollinger_bands(prices)
    rsi = self._calculate_rsi(prices)

    # Calculate Bollinger Band position (-1 to 1)
    band_range = upper_band - lower_band
    if band_range == 0:
        bb_position = 0
    else:
        bb_position = (current_price - middle_band) / (band_range / 2)
        bb_position = np.clip(bb_position, -1, 1)

    # Normalize RSI
    rsi_normalized = (rsi - 50) / 50

    # Combine signals, weight: 60% Bollinger, 40% RSI
    signal = -(0.6 * bb_position + 0.4 * rsi_normalized)

    return signal
```

of this model is to buy stocks when they are oversold and sell them when they are overbought. For instance, the model combines two technical indicators: Bollinger Bands (measuring how far the current price deviates from its <20>-day moving average) and RSI (measuring momentum over 14 days). The signal composts of the two indicators is then weighted and used to decide whether to sell, trim or buy.

For the detailed trading strategy, we can split it into three stages. First, it calculates signal strength for all stocks and immediately exits any positions with signals below <-0.3> (strongly overbought), converting them entirely to cash. Second, it identifies buy candidates—stocks with signals above <0.7> (strongly oversold)—and if none exist, the function returns without trading. Third, it performs portfolio rebalancing by calculating each candidate's target allocation proportional to its signal strength (using <95%> of total portfolio value), then compares the target to current holdings. If underweight by more than one share's worth, it buys more shares; if overweight, it trims the position.

```python
def trade(self):
    signals = {}

    for symbol in self.symbols:
        signal = self.models[symbol].get_signal_strength(self.windows[symbol])
        signals[symbol] = signal

    # Sell overbought positions
    for symbol, count in list(self.counts.items()):
        if count > 0 and signals[symbol] < -0.3:  # Strong sell signal
            value = self.prices[symbol] * count
            self.cash += value * (1 - self.charge)
            print(f"SELL {symbol}: {count} shares @ ${self.prices[symbol]:.2f}, Signal={signals[symbol]:.3f}")
            self.counts[symbol] = 0

    # find oversold candidates
    candidates = []
    signal_sum = 0.0
    for symbol, signal in signals.items():
        if signal > 0.7:  # Oversold threshold
            candidates.append((symbol, signal))
            signal_sum += signal

    if signal_sum == 0:
        return  # No buying opportunities
```

```python
# Calculate total portfolio value
total_portfolio_value = self.cash
for symbol in self.symbols:
    total_portfolio_value += self.counts[symbol] * self.prices[symbol]

# Invest
investable = total_portfolio_value * 0.95  # Keep 5% cash buffer

'''
There is a problem here that if we have a strong buy signal but currently our portfolio
is filled with weak signal stocks, we are not able to sell them to free up cash for the strong buy.
'''
for symbol, signal in candidates:
    # allocate capital for each symbol based on signal strength
    target_value = (signal / signal_sum) * investable
    current_value = self.counts[symbol] * self.prices[symbol]

    if target_value > current_value + self.prices[symbol]:  # Only buy if difference > 1 share
        # BUY more
        shares_needed = int((target_value - current_value) / self.prices[symbol])
        cost = shares_needed * self.prices[symbol]

        if self.cash >= cost:
            self.cash -= cost
            self.counts[symbol] += shares_needed
            print(f"BUY {symbol}: {shares_needed} shares @ ${self.prices[symbol]:.2f}, Signal={signal:.3f}")

    elif current_value > target_value + self.prices[symbol]:  # Only sell if difference > 1 share
        # TRIM position
        shares_to_sell = int((current_value - target_value) / self.prices[symbol])

        if shares_to_sell > 0:
            proceeds = shares_to_sell * self.prices[symbol] * (1 - self.charge)
            self.cash += proceeds
            self.counts[symbol] -= shares_to_sell
            print(f"TRIM {symbol}: {shares_to_sell} shares @ ${self.prices[symbol]:.2f}, Signal={signal:.3f}")
```

However, as noted in the code comment, this approach has a critical flaw—if strong buy signals

appear but the portfolio holds stocks with weak signals (between <-0.3> and <0.7>), those positions cannot be liquidated to free up cash, potentially blocking the strategy from capitalizing on the strongest opportunities.

For the UCB trader, we also used the same data to mock the stock market. The UCB trader follows the UCB1 algorithm for buying stocks. The UCB1 algorithm is a simple algorithm used in the Mult-Armed Bandits problem to converge to an arm with the highest empirical mean.

The UCB1 algorithm [edit]

UCB1, the original UCB method, maintains for each arm i:

- the empirical mean reward $\hat{\mu}_i$,
- the count n_i of times arm i has been played.

At round t, it selects the arm maximizing:

$$\mathrm{UCB1}_i(t) = \hat{\mu}_i + \sqrt{\frac{2\ln t}{n_i}}$$

Arms with $n_i = 0$ are initially played once. The bonus term $\sqrt{2\ln t/n_i}$ shrinks as n_i grows, ensuring exploration of less-tried arms and exploitation of high-mean arms.[1]

In our case, each stock represents an arm the trader pulls (buy) on any given trading day. The UCB trader chooses the stock it wants to buy by calculating the UCB1 index for each stock and then choosing the one with the highest UCB1 index.

```python
def trade(self, day):
    if self.last_symbol is not None:
        self.update_ucb_index(self.last_symbol, day)

    # find stock with highest UCB index
    highest_index_symbol = self.symbols[0]
    for symbol in self.symbols:
        if self.ucb_index[symbol] > self.ucb_index[highest_index_symbol]:
            highest_index_symbol = symbol

    # sell previous day's holdings unless they are the same as the current highest index
    stock
    for symbol in self.symbols:
        if self.counts[symbol] > 0:
            if symbol == highest_index_symbol:
                pass  # hold
            else:
                self._sell_all(symbol)

    if self.counts[highest_index_symbol] == 0:
        self._buy_fraction(
            highest_index_symbol, fraction=0.5
        )  # Buy with 50% of available cash

    self.last_symbol = highest_index_symbol
```

Once the UCB trader has identified the stock it wants to buy based on the highest UCB1 index, it sells off all the holdings from the day before and then uses 50% of the remaining cash balance to purchase as many shares of the chosen stock. If it chooses the same stock as the day before then the algorithm doesn't buy/sell any additional shares and simply holds onto the same holdings.

```python
    def _sell_all(self, symbol):
        count = self.counts[symbol]
        if count > 0:
            value = self.prices[symbol] * count
            self.cash += value * (1 - self.charge)
            self.counts[symbol] = 0

    def _buy_fraction(self, symbol, fraction):
        price = float(self.prices[symbol])
        effective_price = price * (1 + self.charge)

        cash_to_spend = self.cash * float(fraction)
        shares = int(cash_to_spend / effective_price)

        if shares < 1:
            shares = int(self.cash // effective_price)

        if shares > 0:
            cost = shares * effective_price
            self.cash -= cost
            self.counts[symbol] += shares
```

In our main files, we use the following to calculate and report the final balance, total return, and sharpe ratio for each of the trading strategies.

```python
print(f"Final Balance: ${trader.balance():.2f}")
print(f"Total Return: {(trader.balance() - cash) / cash * 100:.2f}%")
print(
    f"Sharpe Ratio: {(np.mean(np.diff(balances)) / np.std(np.diff(balances))):.2f}"
)
```

Meeting Minutes (2/5/2026):

Sian had already implemented the trading simulation plumbing as well as a lstm model to trade the stock. Howard and Viraj will add their own trading algorithms to the project to see if they can generate better returns.